AI Assignment 5: Markov Decision Processes (MDP)

Course: MA3206 AI
Assignment: 5

Problem Definition

The assignment models a battery-powered robot via a Discounted Markov Decision Process (MDP). - **State Space (S)**: `{High, Low, Charging}` - **Action Space (A)**: `{Search, Wait}` - **Constraint**: Action `Search` is explicitly prohibited when the robot is in the `Charging` state. - **System Layout**: - Discount Factor (γ): 0.9 - Convergence limit (θ): 1×10^{-6}

Task 1 | Iterative Policy Evaluation

1.1 Methodology

To determine the expected value $V^{\pi}(s)$ of a constant predetermined policy, the Bellman Expectation Operator is recursively applied. This operator takes the direct expected immediate reward and adds the identical sum of discounted expected probabilities leading to the subsequent iteration values.

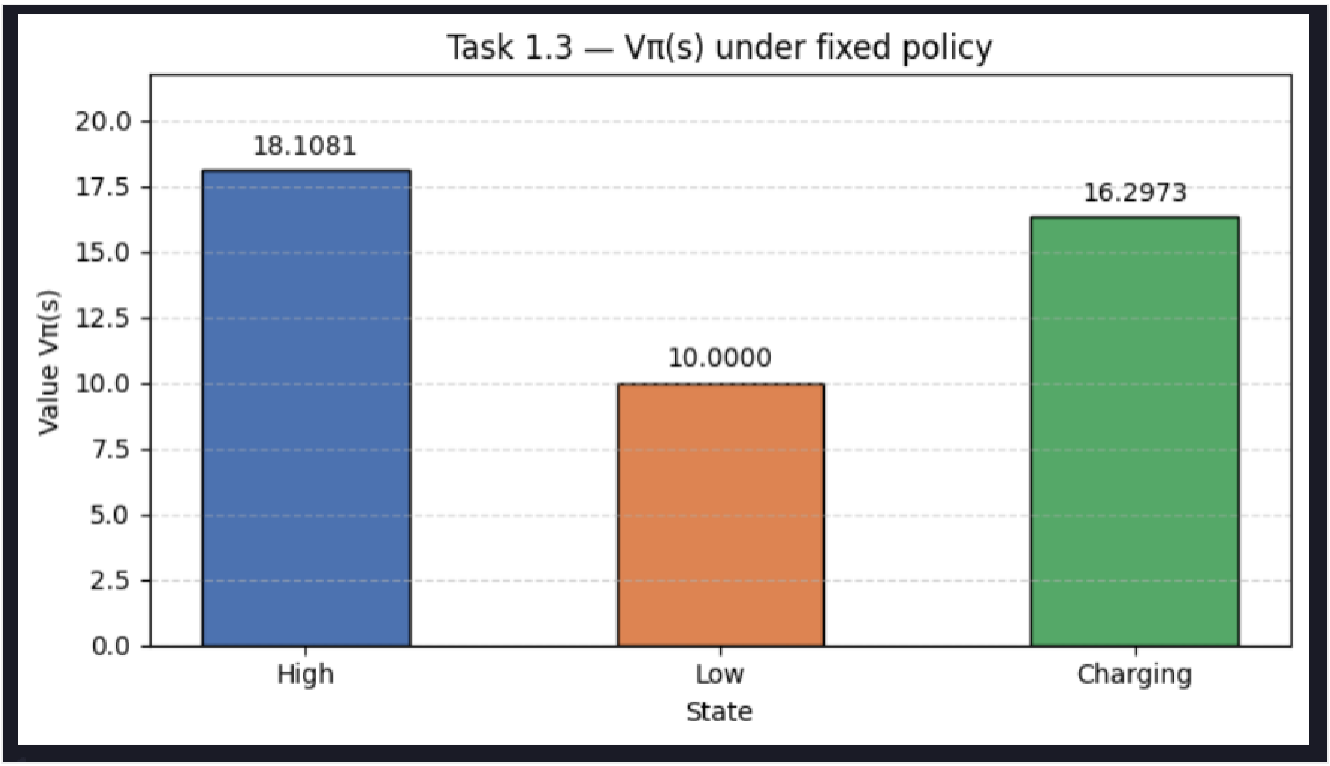
1.2 Results

Evaluating exactly the constrained policy mapping: `High` $\rightarrow$ `Search`, `Low` $\rightarrow$ `Wait`, `Charging` $\rightarrow$ `Wait` The state values cleanly converged to the final results precisely balancing around the mathematically calculated asymptotes for continuous looping bounds.

- $V^{\pi}(\text{High}) \approx 18.108100$
- $V^{\pi}(\text{Low}) \approx 9.999992$
- $V^{\pi}(\text{Charging}) \approx 16.297289$

1.3 Graphical Output

Below is the rendered bar-chart demonstrating the specific expected return metric of each state locked under the predefined π:



Task 2 | Value Iteration

2.1 Methodology

Unlike Task 1, Value Iteration mathematically scours the environment to unearth the globally optimum True $V*$ bounds independently of any locked policy. This runs identically sweeping the space using the Bellman Optimality formula ensuring any maximum Q-value per deterministic sweep drives the bounds upwards. In code, the explicit Q-value corresponding to the forbidden `Charging` + `Search` mapping is firmly trapped to $-\infty$, blocking analytical deviation.

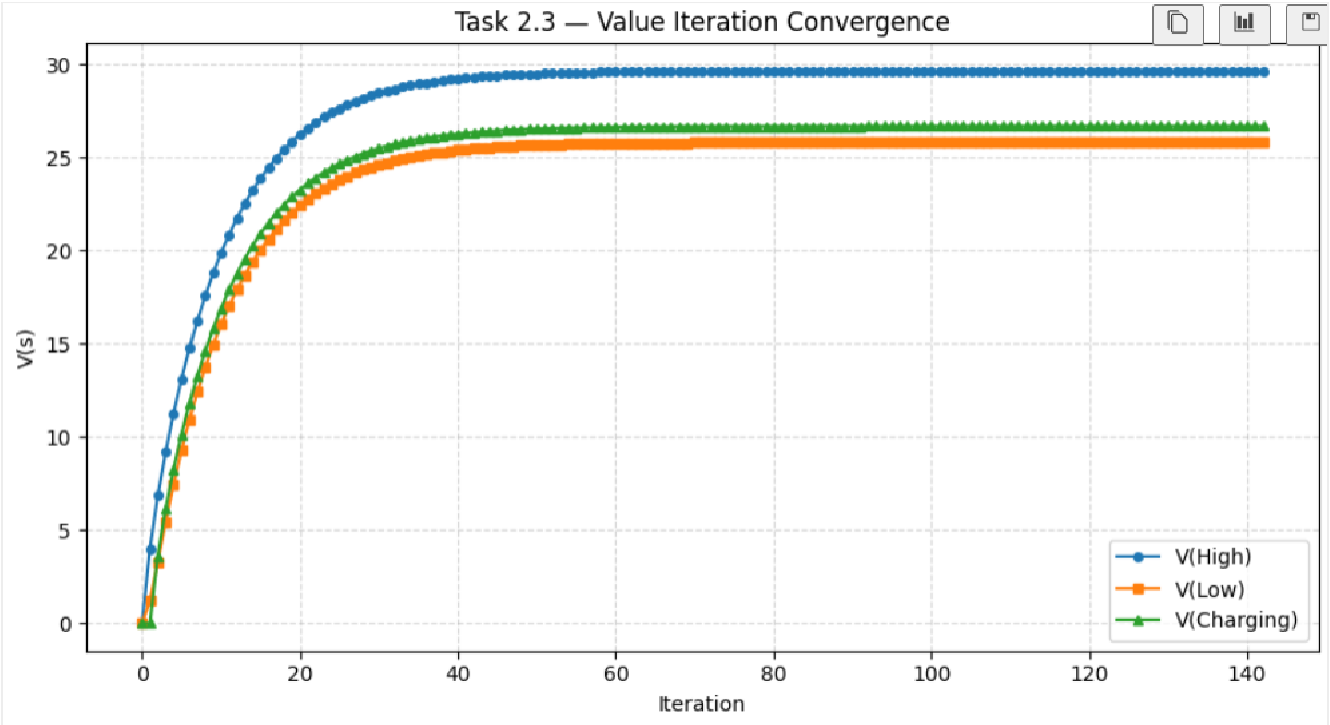
2.2 Results

The iteration algorithm steadily gravitated sequentially, outputting optimal $V*$: - $V*(\text{High}) \approx 29.643827$ - $V*(\text{Low}) \approx 25.808210$ - $V*(\text{Charging}) \approx 26.679443$

Following this computation, pulling the single optimum deterministic $argmax(Q)$ loop constructs the absolute Optimal Policy ($\pi*$): - $\pi*(\text{High}) = \text{Search}$ - $\pi*(\text{Low}) = \text{Search}$ - $\pi*(\text{Charging}) = \text{Wait}$

2.3 Graphical Output

The progression dynamics over multiple computational iteration steps trace the gradual but exponential discovery of maximum long-term bounds.



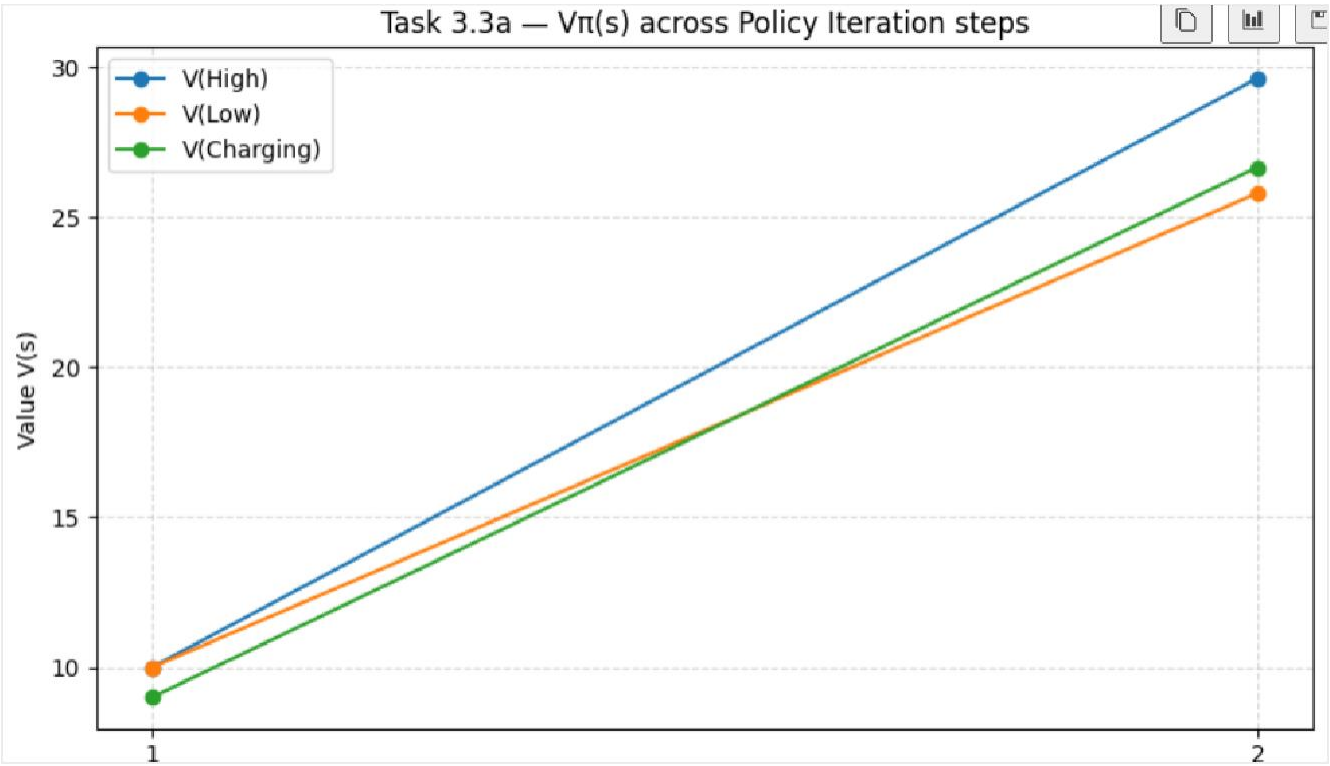
Task 3 | Policy Iteration

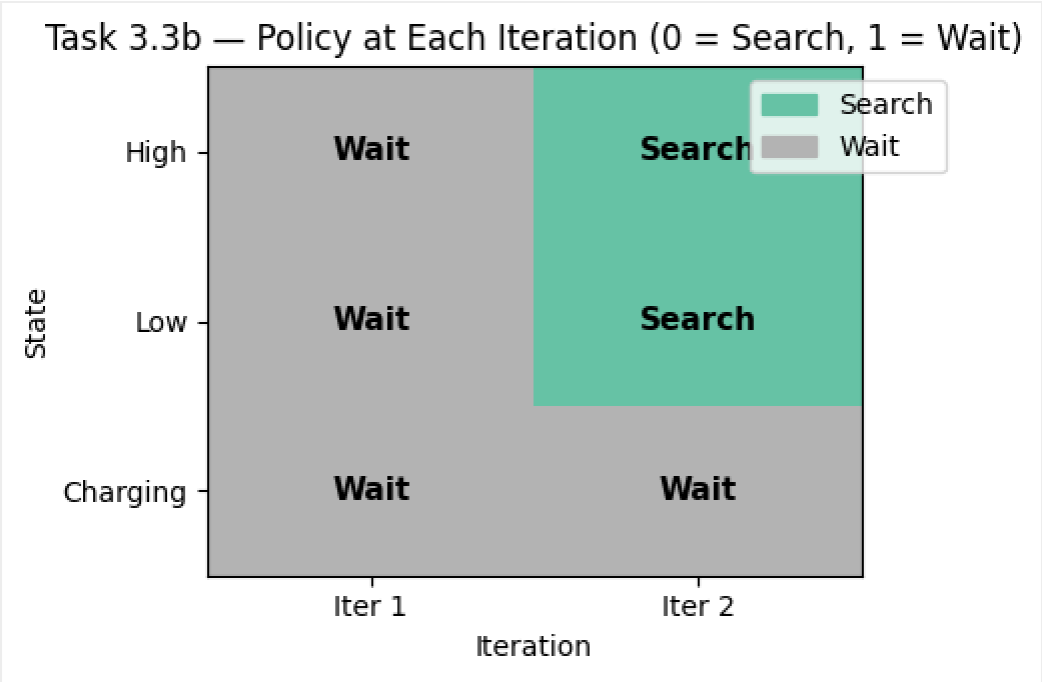
3.1 & 3.2 Methodology

Policy Iteration cycles forcefully across two separate rigid algorithmic phases. The **Evaluation Iteration** forces discovery of true limits associated directly within the bound of the presently enforced active subset. Second, the **Improvement Phase** scans across that fully-evaluated matrix greedily grabbing single action-value offsets to generate entirely new strict policies. This completely eliminates random walks, locking convergence deterministically in mere structural bounds.

3.3 Graphical Output

The algorithm discovered the true identical analytical optimal policy from Value Iteration `[Search, Search, Wait]` after evaluating the states recursively just 2 structural constraint boundaries (outer steps).





Task 4 | Analysis and Interpretation

4.1 Comparing Value Iteration vs. Policy Iteration

- **Convergence Speed**: Policy Iteration converges significantly faster regarding outer structural iterations because verifying policies absolutely limits greedy mapping leaps natively.

- **Iteration Differences**: Value Iteration runs vast quantities of simple optimization scalar sweeps, whereas Policy Iteration takes drastically fewer loops by enforcing extensive mathematical evaluation matrices fully inside each improvement cycle gap.

4.2 Convergence Behaviour

- **Changes Over Time**: As illustrated in the plotted Value Iteration outputs, $V(\text{High})$ quickly inflates because of the inherent $+4$ direct payout. Subsequently, $V(\text{Charging})$ ascends linearly strictly scaling from the inevitable transition returning the system smoothly to `High`.
- **Update Distinction**: Value iteration acts aggressively, finding the strict true max mathematically accessible at that moment. Conversely, policy evaluation inherently anchors upon the statistical probability of a rigidly fixed command array blindly without bias toward optimal exploration.

4.3 Optimal Policy Interpretation

- **Optimum Mapping**: The extracted optimal limits point definitively to `High` $\rightarrow$ `Search`, `Low` $\rightarrow$ `Search`, and linearly constrain `Charging` $\rightarrow$ `Wait`.
- **Intuition**: When robust (`High`), searching guarantees enormous output value efficiently. Likewise, despite the catastrophic battery draining crash odds, aggressively searching while running `Low` yields expected Q-Value boundaries exponentially wider statistically than the marginal bounds of purely choosing `Wait`. Disallowing searching while charging securely traps that element to the necessary looping behavior.

4.4 Practical Engineering Insight

- **Robotics Usefulness**: Hardware sensors mathematically mapping MDPs to discrete operational routines naturally simulate effective biological "forage, exploit, crash, and return-to-base" foraging networks. Rather than expending heavy compute on dynamic runtime analysis arrays, systems seamlessly run static hard-programmed action lookup matrix dictionaries `[Search, Search, Wait]` continuously operating across extreme timeframes limitlessly.